Each of these patterns involves a trade-off. They can lower operational complexity, latency, or deployment overhead, but they come at the expense of consistency and visibility. Table 11.2 compares these four methods.

Table 11.2 Other API authorization enforcement patterns

Pattern	Where enforced	Grain	Typical use
In-process	Inside the service code or via a local Cedar agent	Fine	Low-latency, performance-sensitive services
mTLS or network controls	Connection layer (certs, firewall, NetworkPolicy)	Coarse	Baseline guardrails: which services may talk to each other
Event or broker	Message bus (Kafka, NATS, SQS)	Mostly coarse	Publish/subscribe control; combine with authorization engine for message-level checks
Direct token exchange	Caller presents short-lived STS token to callee	Usually coarse	Smaller systems or when proxies/mesh aren’t available

At ACME, these additional patterns were used selectively. In-process enforcement appeared only in performance-sensitive services where low latency was essential. mTLS and network controls provided coarse-grained guardrails for which services could talk to each other. Broker enforcement applied to asynchronous pipelines, controlling who could publish or consume messages. Direct token exchange was rarely used, limited to smaller systems outside the gateway or mesh. The common thread was that whenever fine-grained access control was required, the decision logic was always externalized to policy in Cedar.